

AI-Assisted Software Development

Module 16 — Business AI in Practice · Notebook 51

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

Contents

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

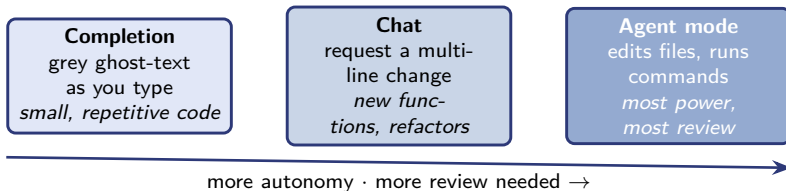
The modern IDE landscape (2026)

Tool	AI assistance	Best for
VS Code + Copilot	inline completion + chat	broad default; Jupyter support
Cursor	chat-first; multi-file refactors	agentic edits, big rewrites
PyCharm + JetBrains AI	completion + assistant panel	large, refactor-heavy projects

Recommendation for beginners

VS Code + Copilot or **Cursor** — the most forgiving combinations to learn the workflow on.

Three ways the assistant helps — rising autonomy



The more the assistant does on its own, the more your review discipline matters (Module 10 — agents, tools & MCP — builds on Agent mode).

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

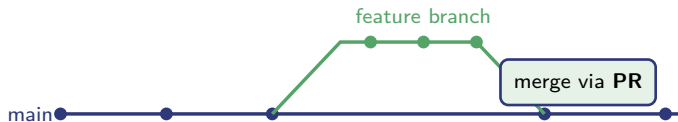
Git — nine commands cover 90 % of daily work

Command	What it does
<code>status / diff</code>	see what changed
<code>add / commit</code>	stage and snapshot
<code>pull / push</code>	sync with the remote
<code>checkout -b / checkout</code>	make / switch a branch
<code>merge</code>	combine a branch back in

Common beginner mistake

Force-pushing over the main branch destroys history others built on. Force-push only your *own* feature branches.

Branches and pull requests — the unit of professional work



A good PR

short title · a 2-paragraph *what* / *why* / *tested* · **small scope** (30 lines reviewed in 5 min) · tests or a screenshot.

AI-generated PRs are no exception — arguably *more* important to keep small.

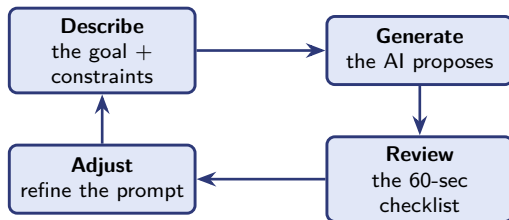
Outline

- 1 IDEs
- 2 Git
- 3 Prompting**
- 4 Review
- 5 When not

Five prompt patterns for code

- ➊ **Give it what it can't see** — the surrounding code, the schema, the error.
- ➋ **Ask for one thing**, explicitly — one change per turn, then iterate.
- ➌ **Be specific about constraints** — version, libraries, style, performance, error semantics.
- ➍ **Ask for tests** — “add a pytest for the empty-input case.”
- ➎ **Ask for alternatives when designing** — “three storage options with trade-offs.”

Prompting is a loop, not a one-shot



Each turn is small and reviewable. You stay the author — the assistant drafts; you decide.

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review**
- 5 When not

Four failure modes of AI-generated code

A — Hallucinated APIs

calls a function / argument that does not exist

B — Plausible but wrong

looks right; the logic is subtly off

C — Architectural drift

ignores the codebase's patterns

D — Silent over-confidence

“safely” swallows errors, hides failure

The 60-second review checklist

Run this on every AI-generated change before you accept or merge

- ❶ **Does it run?** Even just the import line.
- ❷ **Are there tests?** If not, ask for them.
- ❸ **Does it use our libraries / patterns?** Grep for one similar function.
- ❹ **Does it swallow errors?** Search the diff for `except Exception, or {}`, `pass`.
- ❺ **Does it do exactly what was asked** — no more, no less?

60 seconds; catches $\approx 80\%$ of failures. The rest is what real testing and code review are for.

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

When *not* to use the AI assistant

Mental model

AI assistants are most valuable when you *could* write the code yourself but would rather not. They are least valuable when you don't yet understand the problem.

- Skip it when the task is to *learn* the concept — write the first version yourself.
- Skip it for security- / correctness-critical logic you can't fully review.
- **Companion notebook:** hands-on Git, PR, and review exercises are in **Notebook 51** — recommended *early* so the workflow pays off everywhere.